

# GTP4: Programación en C++ para Ciencia e Ingeniería

## Ejercicios

|                                            |          |
|--------------------------------------------|----------|
| <b>Estanislao Claucich</b>                 | <b>1</b> |
| Ejercicio 1: Formas geométricas . . . . .  | 1        |
| Ejercicio 2: Vectores generales . . . . .  | 9        |
| Ejercicio 3: tic-tac-toe general . . . . . | 14       |

## Estanislao Claucich

A continuación se presentan las soluciones a los ejercicios propuestos en el GTP4.

### Ejercicio 1: Formas geométricas

#### Consigna

Implementar la clase `geoshape_t` que represente formas geométricas en 2D. La clase debe incluir métodos para calcular el área, el centro de gravedad y la inercia polar de la forma. Se deben implementar al menos las siguientes formas: círculo, cuadrado, elipse y pentágono regular. A su vez, implementar funciones para calcular la intersección, unión y diferencia entre dos de estas figuras.

#### La clase abstracta `geoshape_t`

El núcleo de la resolución de este ejercicio se basa en explotar el concepto de **polimorfismo**. Se definió una clase puramente virtual `geoshape_t` que expone una interfaz común para todas las figuras geométricas 2D:

```

1 class geoshape_t {
2     public:
3         // Bounding box
4         virtual void bbox(vector<double> &bb)=0;
5
6         // Is x inside shape?
7         virtual bool inside(const vector<double> &x)=0;
8 };

```

Cualquier figura gráfica deberá implementar estos dos métodos: **bbox** (que retorna los límites del eje cartesiano rectangular que encierran a la figura) e **inside** (que determina si un punto (x, y) pertenece o no al interior de la misma).

### Integración numérica para el cálculo de propiedades

La principal ventaja del polimorfismo es que nos permite escribir funciones de cálculo genéricas que reciben una referencia a **geoshape\_t** sin importar qué figura concreta se esté evaluando. Generando una grilla 2D (con un total configurable de particiones,  $N = 500$ ) en el dominio delimitado por el **bbox**, es posible aproximar cálculos complejos como el área, el centro de gravedad y la inercia polar contando y acumulando únicamente aquellos puntos de la grilla donde el método **g.inside(pt)** arroje **true**.

Como ejemplo, el cálculo del área se implementa iterando sobre esta grilla y acumulando los diferenciales  $dx \cdot dy$ :

```

1 double area(geoshape_t &g) {
2     vector<double> bb;
3     g.bbox(bb);
4
5     int N = 500; // Resolución de nuestra grilla de integración
6     double dx = (bb[2] - bb[0]) / N;
7     double dy = (bb[3] - bb[1]) / N;
8
9     double a = 0.0;
10    vector<double> pt(2);
11    for (int i = 0; i < N; ++i) {
12        pt[0] = bb[0] + (i + 0.5) * dx;
13        for (int j = 0; j < N; ++j) {
14            pt[1] = bb[1] + (j + 0.5) * dy;
15            if (g.inside(pt)) {
16                a += dx * dy;

```

```

17     }
18 }
19 }
20 return a;
21 }

```

De manera análoga, para el centroide (`grav_center`) y la inercia polar (`inertia`) sólo cambian los factores a multiplicar por cada paso incremental del área evaluada:

```

1 vector<double> grav_center(geoshape_t &g) {
2     vector<double> bb;
3     g.bbox(bb);
4
5     int N = 500;
6     double dx = (bb[2] - bb[0]) / N;
7     double dy = (bb[3] - bb[1]) / N;
8
9     double a = 0.0, cx = 0.0, cy = 0.0;
10    vector<double> pt(2);
11    for (int i = 0; i < N; ++i) {
12        pt[0] = bb[0] + (i + 0.5) * dx;
13        for (int j = 0; j < N; ++j) {
14            pt[1] = bb[1] + (j + 0.5) * dy;
15            if (g.inside(pt)) {
16                double dA = dx * dy;
17                a += dA;
18                cx += pt[0] * dA;
19                cy += pt[1] * dA;
20            }
21        }
22    }
23    return {cx / a, cy / a};
24 }
25
26 double inertia(geoshape_t &g) {
27     vector<double> cg = grav_center(g);
28     vector<double> bb;
29     g.bbox(bb);
30
31     int N = 500;
32     double dx = (bb[2] - bb[0]) / N;
33     double dy = (bb[3] - bb[1]) / N;

```

```

34
35     double I = 0.0;
36     vector<double> pt(2);
37     for (int i = 0; i < N; ++i) {
38         pt[0] = bb[0] + (i + 0.5) * dx;
39         for (int j = 0; j < N; ++j) {
40             pt[1] = bb[1] + (j + 0.5) * dy;
41             if (g.inside(pt)) {
42                 double r2 = pow(pt[0] - cg[0], 2) + pow(pt[1] - cg[1], 2);
43                 I += r2 * (dx * dy);
44             }
45         }
46     }
47     return I;
48 }

```

## Las figuras base

Con la arquitectura del polimorfismo lista, modelar las figuras se reduce a una tarea muy simple. Figuras primarias como `square_t`, `circle_t` y `ellipse_t` heredan de `geoshape_t` e implementan su propia lógica matemática rápida.

```

1  class circle_t : public geoshape_t {
2      double cx, cy, radius;
3  public:
4      circle_t(double cx, double cy, double r) : cx(cx), cy(cy), radius(r) {}
5
6      void bbox(vector<double> &bb) override {
7          bb = {cx - radius, cy - radius, cx + radius, cy + radius};
8      }
9
10     bool inside(const vector<double> &x) override {
11         return pow(x[0] - cx, 2) + pow(x[1] - cy, 2) <= radius * radius;
12     }
13 };
14
15 class circle_t : public geoshape_t {
16     double cx, cy, radius;
17 public:
18     circle_t(double cx, double cy, double r) : cx(cx), cy(cy), radius(r) {}
19

```

```

20 void bbox(vector<double> &bb) override {
21     bb = {cx - radius, cy - radius, cx + radius, cy + radius};
22 }
23
24 bool inside(const vector<double> &x) override {
25     return pow(x[0] - cx, 2) + pow(x[1] - cy, 2) <= radius * radius;
26 }
27 };
28
29 class ellipse_t : public geoshape_t {
30     double cx, cy, rx, ry;
31 public:
32     ellipse_t(double cx, double cy, double rx, double ry)
33         : cx(cx), cy(cy), rx(rx), ry(ry) {}
34
35     void bbox(vector<double> &bb) override {
36         bb = {cx - rx, cy - ry, cx + rx, cy + ry};
37     }
38
39     bool inside(const vector<double> &x) override {
40         return pow((x[0] - cx) / rx, 2) + pow((x[1] - cy) / ry, 2) <= 1.0;
41     }
42 };

```

En el caso particular de polígonos convexos (`convex_polygon_t`), la lógica de `inside()` requiere utilizar el **producto cruz 2D** entre el punto analizado y las sucesivas aristas dirigidas del polígono, para validar geométricamente que este punto recaiga todo el tiempo sobre el mismo semiplano proyectado:

```

1 class convex_polygon_t : public geoshape_t {
2     vector<double> pts;
3 public:
4     convex_polygon_t(const vector<double> &xj) : pts(xj) {}
5
6     void bbox(vector<double> &bb) override {
7         if (pts.empty()) return;
8         bb = {pts[0], pts[1], pts[0], pts[1]};
9         for (size_t i = 2; i < pts.size(); i += 2) { // Iterar sobre los puntos x, y
10             if (pts[i] < bb[0]) bb[0] = pts[i];
11             if (pts[i] > bb[2]) bb[2] = pts[i];
12             if (pts[i+1] < bb[1]) bb[1] = pts[i+1];
13             if (pts[i+1] > bb[3]) bb[3] = pts[i+1];

```

```

14     }
15 }
16
17 bool inside(const vector<double> &x) override {
18     if (pts.size() < 6) return false; // Todo polígono debe tener al menos 3 vértices
19     int sign = 0;
20     size_t n = pts.size() / 2;
21     // Producto cruz
22     for (size_t i = 0; i < n; ++i) {
23         double x1 = pts[2*i];
24         double y1 = pts[2*i + 1];
25         double x2 = pts[(2*i + 2) % (2*n)];
26         double y2 = pts[(2*i + 3) % (2*n)];
27
28         double dx = x2 - x1;
29         double dy = y2 - y1;
30         double cross = dx * (x[1] - y1) - dy * (x[0] - x1);
31
32         if (cross != 0) {
33             int param_sign = (cross > 0) ? 1 : -1;
34             if (sign == 0) sign = param_sign;
35             else if (sign != param_sign) return false;
36         }
37     }
38     return true;
39 }
40 };

```

## Operaciones booleanas entre figuras

Este diseño orientado a objetos posibilita armar operaciones Booleanas que compongan geometrías a partir de otras, combinándolas para calcular la solución gráfica con un bajo costo arquitectural. Para ello creamos rutinas Booleanas `intersection_t`, `union_t` y `difference_t`.

Dichas clases almacenan en su interior una variable `vector<geoshape_t*> shapes;` y consiguen un comportamiento acoplado condicionando la regla del `inside()`. Analicemos el caso de la Intersección: para considerarse partícipe del sistema central, el punto evaluado debe recaer dentro del interior de la totalidad de las piezas referidas.

```

1 class intersection_t : public geoshape_t {
2 public:
3     vector<geoshape_t*> shapes;

```

```

4     void bbox(vector<double> &bb) override {
5         /* Se obtiene solapando y achicando recursivamente las cajas de las figuras... */
6     }
7     bool inside(const vector<double> &x) override {
8         if (shapes.empty()) return false;
9         for (auto* shape : shapes) {
10            // El punto debe existir en TODAS las sub-figuras
11            if (!shape->inside(x)) return false;
12        }
13        return true;
14    }
15 };

```

Para comprobar la correcta implementación, se instanciaron 4 figuras diferentes sencillas donde sabemos previamente el resultado exacto de las distintas operaciones:

```

1 // 1. Verificar Cuadrado (Lado L=2, centrado en el origen)
2 square_t sq(-1.0, -1.0, 1.0, 1.0);
3 double sq_area_exact = 2.0 * 2.0;
4 double sq_inertia_exact = (2.0 * pow(2.0, 3) / 12.0) + (2.0 * pow(2.0, 3) / 12.0);
5
6 // 2. Verificar Círculo (Radio R=2, centrado en el origen)
7 circle_t circ(0.0, 0.0, 2.0);
8 double pi = acos(-1.0);
9 double circ_area_exact = pi * pow(2.0, 2);
10 double circ_inertia_exact = pi * pow(2.0, 4) / 2.0;
11
12 // 3. Verificar Elipse (rx=2, ry=1, centrada en el origen)
13 ellipse_t el(0.0, 0.0, 2.0, 1.0);
14 double el_area_exact = pi * 2.0 * 1.0;
15 // Inercia de la elipse = I_x + I_y = (pi * rx * ry^3 / 4) + (pi * ry * rx^3 / 4)
16 double el_inertia_exact = (pi * 2.0 * pow(1.0, 3) / 4.0) + (pi * 1.0 * pow(2.0, 3) / 4.0);
17
18 // 4. Pentágono regular (R=2, centrado en el origen)
19 vector<double> pent_pts;
20 double R = 2.0;
21 for (int i = 0; i < 5; ++i) { // 5 vértices espaciados 72 grados
22     pent_pts.push_back(R * cos(i * 2.0 * pi / 5.0));
23     pent_pts.push_back(R * sin(i * 2.0 * pi / 5.0));
24 }
25 convex_polygon_t pent(pent_pts);
26

```

```

27 // Fórmulas exactas para un polígono regular de N lados circunscrito en un círculo de radio 1
28 double pent_area_exact = (5.0 / 2.0) * pow(R, 2) * sin(2.0 * pi / 5.0);
29 double pent_inertia_exact = pent_area_exact * pow(R, 2) / 6.0 * (2.0 + cos(2.0 * pi / 5.0));

```

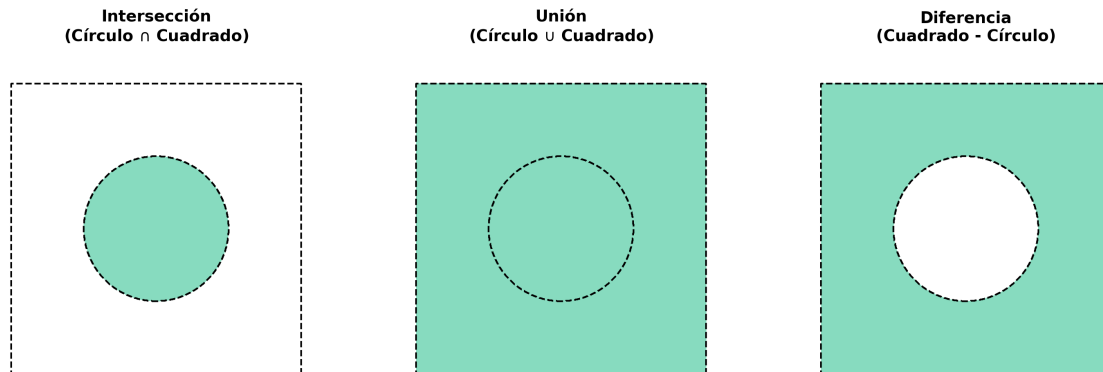
```
bash
```

```

— Cuadrado (L=2) —
Area (Num / Exacta): 4.0000 / 4.0000
Centro de Gravedad: (0.0000, 0.0000) / Exacto: (0.0000, 0.0000)
Inercia (Num / Exacta): 2.6667 / 2.6667
— Circulo (R=2) —
Area (Num / Exacta): 12.5663 / 12.5664
Centro de Gravedad: (0.0000, 0.0000) / Exacto: (0.0000, 0.0000)
Inercia (Num / Exacta): 25.1325 / 25.1327
— Interseccion (Circulo inscrito en Cuadrado) —
Area (Num / Exacta): 3.1416 / 3.1416 (Area del Circulo)
— Union (Circulo inscrito en Cuadrado) —
Area (Num / Exacta): 16.0000 / 16.0000 (Area del Cuadrado)
— Pentagono Regular (R=2) —
Area (Num / Exacta): 9.5106 / 9.5106
Centro de Gravedad: (0.0000, 0.0000) / Exacto: (0.0000, 0.0000)
Inercia (Num / Exacta): 20.8986 / 20.8986

```

Se comprueban las operaciones booleanas con un caso simple y controlado. Un círculo de radio 1 inscrito en un cuadrado de lado 4, donde el área de la intersección debe ser igual al área del círculo y el área de la unión debe ser igual al área del cuadrado:



```

1 // Cuadrado de 4x4 centrado en el origen
2 square_t bsq(-2.0, -2.0, 2.0, 2.0);
3 // Círculo de R=1 centrado en el origen

```



```

4  circle_t bcirc(0.0, 0.0, 1.0);
5
6  intersection_t inter;
7  inter.shapes.push_back(&bsq);
8  inter.shapes.push_back(&bcirc);
9
10 union_t uni;
11 uni.shapes.push_back(&bsq);
12 uni.shapes.push_back(&bcirc);
13
14 difference_t diff;
15 diff.shapes.push_back(&bsq);
16 diff.shapes.push_back(&bcirc);

```

bash

```

— Interseccion (Circulo inscrito en Cuadrado) —
Area (Num / Exacta): 3.1418 / 3.1416 (Area del Circulo)
— Union (Circulo inscrito en Cuadrado) —
Area (Num / Exacta): 16.0000 / 16.0000 (Area del Cuadrado)
— Diferencia (Cuadrado menos Circulo) —
Area (Num / Exacta): 12.8589 / 12.8584

```

## Ejercicio 2: Vectores generales

### Consigna

Escribir una clase polimórfica para representar vectores. Se deben implementar los vectores `fullvec_t` (almacena todos los elementos), `stride_t` (almacena un valor inicial, un paso y la cantidad de elementos) y `sparse_t` (almacena la cantidad total de elementos, un vector con los índices de los elementos no nulos y otro vector con los valores correspondientes). Implementar funciones para calcular el número de elementos no nulos, la suma, el máximo y el mínimo de los elementos del vector. Además, implementar una función `reduce` que reciba un vector y una operación asociativa (suma, máximo o mínimo) y devuelva el resultado de aplicar esa operación a todos los elementos del vector.

### La clase abstracta `vector_t`

De manera análoga al ejercicio anterior, definimos una interfaz base abstracta `vector_t`. Su objetivo es ocultar cómo se guardan realmente los elementos en memoria, forzando a que todo

tipo de vector responda únicamente a dos consultas básicas: `size()` (obtener la longitud del vector) y el sobrecargado `operator[]` (obtener el valor alojado en la posición `i`).

```
1 class vector_t {
2     public:
3         virtual int size() = 0;
4         virtual double operator[ ](int i) = 0;
5 };
```

Con esta interfaz, se implementaron rutinas genéricas (`non_null`, `sum`, `max`, `min`) que operan ciegamente sobre referencias `vector_t &v`. Iterando hasta su respectivo `.size()`, y utilizando su sobrecarga dinámica de indexado `v[i]`. De esta forma, podemos calcular estas propiedades sin importar cuál es la naturaleza de almacenamiento del vector en particular:

```
1 int non_null(vector_t &v) {
2     int count = 0;
3     for (int i = 0; i < v.size(); ++i) {
4         if (v[i] != 0.0) count++;
5     }
6     return count;
7 }
8
9 double sum(vector_t &v) {
10     double total = 0.0;
11     for (int i = 0; i < v.size(); ++i) {
12         total += v[i];
13     }
14     return total;
15 }
16
17 double max(vector_t &v) {
18     if (v.size() == 0) return numeric_limits<double>::lowest();
19     double current_max = v[0];
20     for (int i = 1; i < v.size(); ++i) {
21         if (v[i] > current_max) current_max = v[i];
22     }
23     return current_max;
24 }
25
26 double min(vector_t &v) {
27     if (v.size() == 0) return numeric_limits<double>::max();
28     double current_min = v[0];
```

```

29     for (int i = 1; i < v.size(); ++i) {
30         if (v[i] < current_min) current_min = v[i];
31     }
32     return current_min;
33 }

```

## Diferentes vectores

Se definieron tres tipos de vectores:

1. **fullvec\_t**: Un vector denso clásico:

```

1  class fullvec_t : public vector_t {
2      private:
3          vector<double> w;
4      public:
5          fullvec_t(int sz) : w(sz, 0.0) {}
6          fullvec_t(const vector<double>& vec) : w(vec) {}
7
8          int size() override {
9              return w.size();
10         }
11
12         double operator[](int i) override {
13             return w[i];
14         }
15 };

```

2. **stride\_t**: No guarda valores individualizados sino su definición matemática secuencial pura (start, inc, size). Resulta sumamente eficiente en memoria, permitiendo emular millones de puntos usando sólo tres variables escalares con una resolución al vuelo:

```

1  class stride_t : public vector_t {
2      private:
3          double start;
4          double inc;
5          int sz;
6      public:
7          stride_t(double start, double inc, int sz) : start(start), inc(inc), sz(sz) {}
8
9          int size() override {

```

```

10         return sz;
11     }
12
13     double operator[](int i) override {
14         return start + i * inc;
15     }
16 };

```

3. **sparse\_t**: Vector raro. Está diseñado para conjuntos masivos compuestos mayoritariamente por ceros absolutos, para ahorrar RAM. En lugar de un arreglo denso masivo, conserva dos arreglos limitados a la par: los índices afectados (**indx**) y sus respectivos valores numéricos (**vals**). Al pedir una posición no listada, asume implícitamente 0.0.

```

1  class sparse_t : public vector_t {
2      private:
3          int sz;
4          vector<int> indx;
5          vector<double> vals;
6      public:
7          sparse_t(int sz, const vector<int>& indx, const vector<double>& vals)
8              : sz(sz), indx(indx), vals(vals) {}
9
10         int size() override {
11             return sz;
12         }
13
14         double operator[](int i) override {
15             // Buscamos si el índice existe en los valores definidos
16             for (size_t k = 0; k < indx.size(); ++k) {
17                 if (indx[k] == i) {
18                     return vals[k];
19                 }
20             }
21             // Si es un índice no registrado, implícitamente vale nulo (0.0)
22             return 0.0;
23         }
24 };

```

Vemos una simple prueba de la funcionalidad implementada:

```

1 fullvec_t fv({1.0, 0.0, -3.0, 4.0, 0.0});
2 // 0, 0.5, 1.0, 1.5, 2.0
3 stride_t sv(0, 0.5, 5);
4 // Vector de 100 ptos, solo 3 elementos no nulos
5 sparse_t spv(100, {5, 50, 99}, {-3.0, 0.0, 8.0});

```

```
bash
```

```

— Full Vector —
Non nulls: 3 | Sum: 2 | Max: 4 | Min: -3
— Stride Vector —
Non nulls: 4 | Sum: 5 | Max: 2 | Min: 0
— Sparse Vector —
Non nulls: 2 | Sum: 5 | Max: 8 | Min: -3

```

## Reducción funcional y clases asociativas

Para generalizar al máximo operaciones repetitivas (como la suma o la búsqueda del valor máximo en una sola función universal **reduce**), se empleó la inyección de dependencias mediante la clase abstracta **assoc\_t**. Esta clase porta un elemento neutro **null()** y una función binaria asociativa **g(x, y)**.

```

1 class assoc_t {
2     public:
3         virtual double g(double x,double y)=0;
4         virtual double null()=0;
5 };
6
7 class sum_t : public assoc_t {
8     public:
9         double g(double x,double y) override { return x+y; }
10        double null() override { return 0; }
11 };
12
13 class max_t : public assoc_t {
14     public:
15         double g(double x,double y) override { return (x > y) ? x : y; }
16        double null() override { return numeric_limits<double>::lowest(); }
17 };
18
19 class min_t : public assoc_t {

```

```

20     public:
21         double g(double x, double y) override { return (x < y) ? x : y; }
22         double null() override { return numeric_limits<double>::max(); }
23     };
24
25     double reduce(vector_t &v, assoc_t &op) {
26         if (v.size() == 0) return op.null();
27
28         double result = op.null();
29         for (int i = 0; i < v.size(); ++i) {
30             result = op.g(result, v[i]);
31         }
32         return result;
33     }

```

Pudiendo crear derivados simples como un `sum_t` que sume o un `max_t`, logramos que al instanciar herramientas abstractas sobre esta rutina genérica, se computen los mismos resultados analíticos deseados que antes pero sin necesidad de mantener múltiples ciclos iteradores idénticos recorriendo nuestra memoria.

```
bash
```

```

— Prueba Reduce (Sparse Vector) —
Reduce Sum: 5.0000
Reduce Max: 8.0000
Reduce Min: -3.0000

```

### Ejercicio 3: tic-tac-toe general

#### Consigna

A partir del GTP anterior de tic-tac-toe, generalizar los jugadores a partir de una clase `player_t` que permita simplificar la implementación de la interacción entre jugadores diferentes.

Implementar un sistema de competencia, comenzando desde octavos de final, donde se enfrentarán jugadores aleatorios entre sí, realizando  $N$  partidas entre sí y obteniendo un ganador. Además, sortear las ventajas invirtiendo los turnos iniciales y administrar la memoria borrando a los perdedores en el proceso.

## La clase abstracta `player_t`

Para generalizar el comportamiento de los diferentes agentes virtuales que juegan al tic-tac-toe, se diseñó una clase abstracta pura `player_t` de la cual deban heredar forzosamente todas las estrategias implementadas en módulos anteriores.

Esta interfaz los obliga a tener una variable con un nombre universal y la definición de su lógica de turno subyacente matemáticamente a través del método `move`. Asimismo, declara un destructor `virtual` para habilitar el manejo de la jerarquía polimórfica sin generar fugas de memoria remanentes al tratar de emplear `delete` usando su tipo padre (`player_t`):

```
1 class player_t {
2     protected:
3         string name;
4     public:
5         player_t(string id = "") : name(id) {}
6         virtual int move(board_t &b, int me) = 0;
7         virtual string label() { return name; }
8 };
```

Con esto, las heurísticas previas (`dumb_t`, `random_t`, `agresive_t`, `minimax_t` y `rule_based_t`) asumen prefijos de registro para ser fácilmente identificables (e.g. `minimax_14`) y extienden de ella.

## Playoffs

El modelo orientado a objetos (donde tácticas concretas devienen abstractamente un simple `player_t*`) consigue que a la función principal encargada de las mecánicas no le importe a quiénes maneja:

```
1 vector<int> match(player_t &playX, player_t &playO, int n_matches=1, bool verbose=true)
```

Seguidamente, se formuló un sistema principal de `playoffs` donde competirán diferentes tipos de jugadores entre sí y se obtendrá un campeón. Como el juego tiene una pequeña ventaja por quien comienza, se juegan  $N$  partidas con cada jugador iniciando primero, para luego sumar los resultados y determinar un ganador estadístico:

```
1 // Partidas donde p1 juega primero y donde p2 juega primero
2 vector<int> res1 = match(*p1, *p2, matches_per_round, false);
3 vector<int> res2 = match(*p2, *p1, matches_per_round, false);
4
```

```

5 // Sumatoria total de triunfos
6 int p1_wins = res1[0] + res2[1];
7 int p2_wins = res1[1] + res2[0];
8 int ties = res1[2] + res2[2];

```

En caso de que persista un total nivelado tras todas las rondas, el ganador se rifará aleatoriamente (`rand() % 2 == 0`). Además, tras decidir quién avanza a la siguiente ronda, el puntero heap del contrincante perdedor se purga dinámicamente llamando a su destructor (vía `delete p1/p2`) para no colapsar la RAM de cara a fases posteriores.

## Simulación de Torneo Aleatorio

En el `main`, se instancian 16 punteros conformando los octavos de final. Cada uno de estos agentes está designado ciegamente entre los 5 algoritmos disponibles. Puesto que los mejores agentes se abren paso inevitablemente por el filtro estadístico, es natural ver chocar a los `minimax` cerca de la cima para un desempate numérico total entre 20 simulaciones simultáneas probados como Cruz y luego otras 20 como Círculo:

```

1 int main() {
2     srand(time(NULL));
3     vector<player_t*> tournament;
4
5     // Octavos de final: 16 jugadores aleatorios
6     for(int i = 1; i <= 16; i++) {
7         int r = rand() % 5;
8         string id = to_string(i);
9         if (r == 0) tournament.push_back(new dumb_t(id));
10        else if (r == 1) tournament.push_back(new random_t(id));
11        else if (r == 2) tournament.push_back(new agresive_t(id));
12        else if (r == 3) tournament.push_back(new minimax_t(id));
13        else tournament.push_back(new rule_based_t(id));
14    }
15
16    player_t* champion = playoffs(tournament, 10);
17
18    delete champion;
19    return 0;
20 }

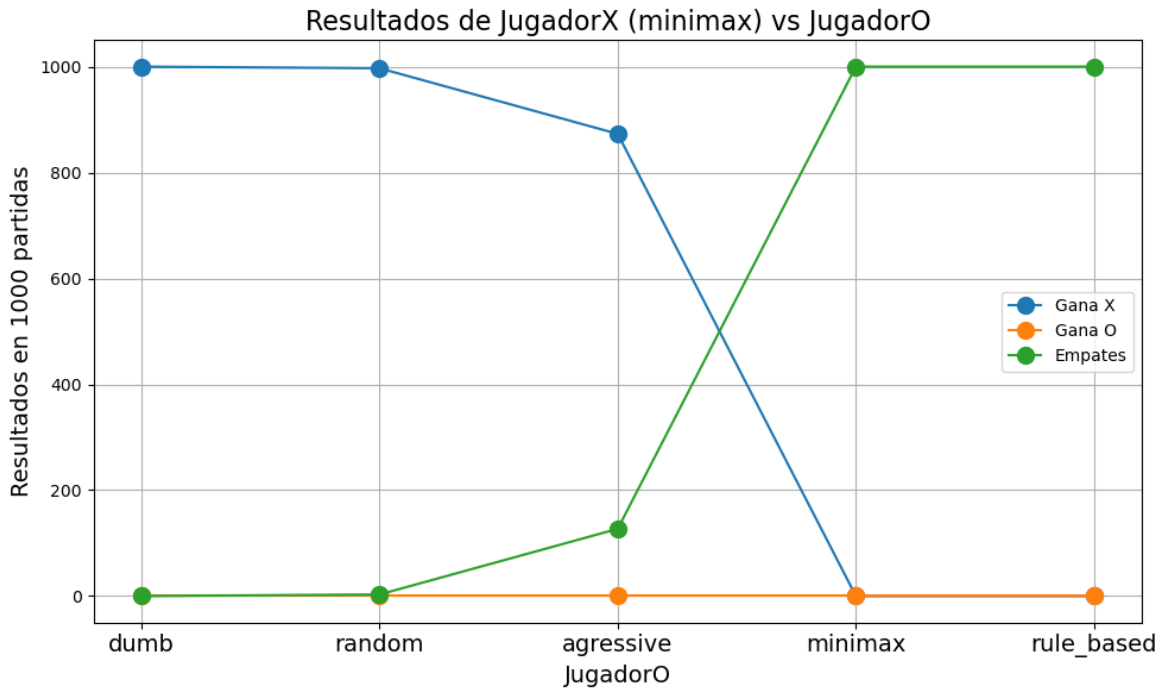
```



```
bash
```

```
=== Ronda 1 (16 jugadores) ===
minimax_1 vs random_2 -> Gana minimax_1 (20-0 y 0 empates)
agresive_3 vs dumb_4 -> Gana agresive_3 (20-0 y 0 empates)
rule_based_5 vs random_6 -> Gana rule_based_5 (18-1 y 1 empates)
minimax_7 vs minimax_8 -> Empate (0-0), desempatando al azar -> minimax_8
dumb_9 vs agresive_10 -> Gana agresive_10 (19-0 y 1 empates)
random_11 vs minimax_12 -> Gana minimax_12 (19-0 y 1 empates)
rule_based_13 vs rule_based_14 -> Empate (0-0), desempatando al azar ->
rule_based_13
dumb_15 vs random_16 -> Gana random_16 (16-1 y 3 empates)
=== Ronda 2 (8 jugadores) ===
minimax_1 vs agresive_3 -> Gana minimax_1 (20-0 y 0 empates)
rule_based_5 vs minimax_8 -> Gana minimax_8 (14-0 y 6 empates)
agresive_10 vs minimax_12 -> Gana minimax_12 (20-0 y 0 empates)
rule_based_13 vs random_16 -> Gana rule_based_13 (17-1 y 2 empates)
=== Ronda 3 (4 jugadores) ===
minimax_1 vs minimax_8 -> Empate (0-0), desempatando al azar -> minimax_8
minimax_12 vs rule_based_13 -> Gana minimax_12 (12-0 y 8 empates)
=== Ronda 4 (2 jugadores) ===
minimax_8 vs minimax_12 -> Empate (0-0), desempatando al azar -> minimax_12
¡El ganador de los playoffs es minimax_12!
```

Siempre termina pasando que a la final llegan dos jugadores `minimax`. Y como vimos en el práctico anterior, cuando se enfrentan dos `minimax` con la misma profundidad de búsqueda, el resultado es un empate técnico (ver figura, punto `minimax`). Por lo tanto, el campeón se define por sorteo puro entre ambos finalistas.



Para hacerlo más interesante, simulamos un torneo donde no pueda haber jugadores `minimax`:

```
bash
```

```
=== Ronda 1 (16 jugadores) ===
random_1 vs random_2 -> Gana random_1 (11-6 y 3 empates)
random_3 vs dumb_4 -> Gana dumb_4 (11-9 y 0 empates)
dumb_5 vs agresive_6 -> Gana agresive_6 (19-0 y 1 empates)
dumb_7 vs dumb_8 -> Empate (10-10), desempatando al azar -> dumb_8
agresive_9 vs dumb_10 -> Gana agresive_9 (17-0 y 3 empates)
agresive_11 vs random_12 -> Gana agresive_11 (17-0 y 3 empates)
dumb_13 vs random_14 -> Gana dumb_13 (11-8 y 1 empates)
random_15 vs random_16 -> Gana random_16 (11-9 y 0 empates)
=== Ronda 2 (8 jugadores) ===
random_1 vs dumb_4 -> Gana dumb_4 (12-7 y 1 empates)
agresive_6 vs dumb_8 -> Gana agresive_6 (18-0 y 2 empates)
agresive_9 vs agresive_11 -> Gana agresive_9 (4-3 y 13 empates)
dumb_13 vs random_16 -> Gana dumb_13 (14-6 y 0 empates)
=== Ronda 3 (4 jugadores) ===
dumb_4 vs agresive_6 -> Gana agresive_6 (18-0 y 2 empates)
agresive_9 vs dumb_13 -> Gana agresive_9 (18-0 y 2 empates)
=== Ronda 4 (2 jugadores) ===
```

agresive\_6 vs agresive\_9 -> Gana agresive\_9 (8-1 y 11 empates)  
¡El ganador de los playoffs es agresive\_9!